Full Stack Developer Assignment

> ***Estimated Time**: 3–5 days*
>
> ***Submission Deadline**: As communicated by the hiring team*
>
> ***Difficulty**: Intermediate*

Overview

We are looking for a Full Stack Developer who can work across the entire stack — from building clean, responsive UIs to designing robust backend APIs and integrating AI/LLM capabilities.

Your task is to build a **Resume Customizer Web App** — a tool that takes a **Job Description (JD)** and a **candidate's resume**, and uses an **LLM** to intelligently tailor the resume to match the JD.

Problem Statement

Job seekers often struggle to customize their resumes for each role they apply to. Build a web application that:

1. Accepts a **Job Description** (pasted as text)
2. Accepts a **Resume** (uploaded as PDF or DOCX)
3. Uses an **LLM** to analyze the JD and customize the resume accordingly
4. Returns a **tailored resume** that the user can view and download

---

Tech Stack Requirements

Layer	Technology	Notes
Frontend	**Next.js** (React)	App Router preferred
Backend	**Python**	FastAPI or Flask
LLM	Any LLM API	Gemini, OpenAI, Groq, etc.
Hosting	Any cloud platform	Vercel, Render, Railway, GCP, AWS, etc.

> *You are free to choose any additional libraries, UI frameworks, or tools you see fit.*

Core Features (Required)

1. Job Description Input

- A text area where the user can **paste the full JD**
- Basic validation (non-empty, minimum length)

2. Resume Upload

- Support **PDF and/or DOCX** file upload
- Show file name and size after upload
- Parse and extract text content from the resume on the backend

3. LLM-Powered Resume Customization

- Send the JD + parsed resume content to an **LLM API**
- The LLM should:
- Rewrite the **summary/objective** to align with the JD
- Highlight **relevant skills** from the resume that match the JD
- Reorder or rephrase **experience bullets** to better fit the JD keywords
- Suggest any **missing keywords** the candidate could add
- Return the customized resume content to the frontend

4. Output Display

- Show the **customized resume** in a clean, readable format on the UI
- Provide a **Download** option (as PDF or DOCX or plain text)

5. Basic Error Handling

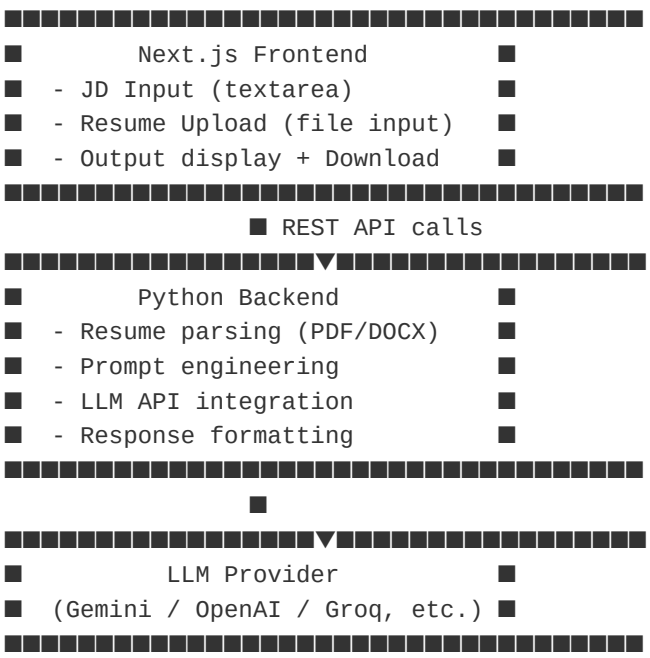
- Handle invalid file types, large files, empty inputs
- Show meaningful error messages to the user
- Handle LLM API failures gracefully

Bonus Features (Optional but Impressive)

- [ ] **Match Score** — Show a % match between the original resume and JD
- [ ] **Diff View** — Highlight what changed between the original and customized resume
- [ ] **Multiple Resume Versions** — Allow users to generate and compare multiple tailored versions
- [ ] **Section-wise customization** — Let users choose which sections to customize (Summary, Skills, Experience, etc.)
- [ ] **Cover Letter Generator** — Generate a cover letter based on the JD and resume
- [ ] **ATS Score** — Estimate how ATS-friendly the customized resume is
- [ ] **Authentication** — Basic user login to save history
- [ ] **Streaming Response** — Stream the LLM output to the UI in real time

---

Architecture Guidelines



Hosting Requirements

- The application **must be deployed and accessible via a public URL**
- Frontend and backend can be hosted on the **same or different platforms**

Suggested free hosting options:

Service	What to host
Vercel	Next.js frontend
Render / Railway	Python backend
Google Cloud Run	Python backend (containerized)
Hugging Face Spaces	Full stack (if using Gradio/FastAPI)

> ***Do not submit a localhost-only project.*** *A live hosted link is mandatory.*

Deliverables

Please submit the following:

1. **GitHub Repository Link**

- Clean, well-organized code
- Separate folders for `frontend/` and `backend/`

- `.env.example` file (never commit real API keys!)

2. **Live Application URL**

- A working, publicly accessible hosted link

3. **README.md** in the repo containing:

- Project overview
- Tech stack used
- LLM provider and model used
- Setup and run instructions (local)
- Architecture diagram or description
- Any known limitations or trade-offs
- (Optional) Screenshots or demo video

Evaluation Criteria

Your submission will be evaluated on the following:

Criteria	Weight	What We Look For
Functionality	30%	Does the core feature work end-to-end?
Code Quality	20%	Clean, readable, maintainable code
LLM Integration	20%	Quality of prompt engineering and output
UI/UX	15%	Intuitive, clean, and responsive design
Deployment	10%	App is live and accessible
Bonus Features	5%	Any extras beyond the core requirements

Tips & Guidelines

- **Prompt Engineering matters** — spend time crafting a good system prompt for the LLM. The quality of the customized resume depends heavily on it.
- **Free LLM APIs** — You can use Google AI Studio for a free Gemini API key, or Groq for free Llama/Mixtral access.
- **Resume Parsing** — Use libraries like `pdfplumber`, `PyMuPDF`, or `python-docx` to extract text from uploaded files.
- **Keep it simple first** — Get the core flow working before adding bonus features.
- **Don't over-engineer** — We value clean, working code over complex architectures.

- **Environment variables** — Never hardcode API keys. Use `.env` files.

FAQ

Q: Can I use a UI component library?

A: Yes — Shadcn/UI, Chakra UI, Tailwind CSS, Ant Design, etc. are all fine.

Q: Which LLM should I use?

A: Your choice. We recommend Gemini (free via Google AI Studio) or Groq (free tier available). Document your choice in the README.

Q: Can I use LangChain or similar frameworks?

A: Yes, you may use LangChain, LlamaIndex, or call the LLM API directly.

Q: What if I can't parse DOCX?

A: PDF support is the minimum requirement. DOCX is a bonus.

Q: Can I use a database?

A: Not required for core functionality, but you may add one for bonus features.

Q: What if I face issues with hosting?

A: Reach out to us before the deadline. We'd rather help you than penalize you for infra issues.

Assignment created by the Engineering Hiring Team